NAZARBAYEV INTELLECTUAL SCHOOL OF SCIENCE AND MATHEMATICS IN KARAGANDY

section: **AI programming**

«Neuromusix: Adaptive music generation using reinforcement learning»

Made by: **Abeuov Yerassyl,**
Grade 11H

Mentor: **Ustinova Lyudmila Valeryevna,**
Teacher of informatics

Karaganda 2026г.

Abstract

The project explores the possibility of generating musical compositions using artificial intelligence (reinforcement learning) adaptively to the user's musical preferences, with potential positive effects on user well-being. The field of AI music is actively developing in 2026. The hypothesis is that by using a user-centric reinforcement learning mechanism in music generation, considering the dimensions of energy, depth, complexity, openness, and structure, a program capable of generating melodies according to the user's preferences and mood is realized.
The goal of the project is to create a program that will generate musical compositions based on user characteristics, with export, and which will also learn in real time based on user ratings. A strict methodology was followed: competitor research for 2026, research into music theory and psychology, selection of a programming language and libraries, planning of the program architecture, and implementation of the program code, simultaneously creating a suitable GUI. All research and programming were completed independently. The result was a program with a user-friendly GUI that generates music based on the user's preferences by using reinforcement learning (AI principle) and adapts to them, plays the melody on various instruments to create an orchestral piece, and displays the notes being played in real time on a piano keyboard. The hypothesis was confirmed: adaptation is a feasible path for the future of computer music. The project can potentially provide musical medicine, positively impacting the well-being of users, and inspire musicians in the creation of their own songs.

Introduction

Music is an important part of humanity. Sound is a universal language for exploring mood, as well as enhancing human well-being. In history, the music industry has evolved rapidly from the creation of musical instruments (flute, piano, organ, violin) to the creation of the digital audio workstations (DAW) which allows anyone to create unique melodies on a computer synthesizer. Music has now entered a new phase with artificial intelligence. Yet, while previous progress just facilitated music composition, AI is aiming to replace composers entirely.

In 2026, the music industry is a crucial and profitable part of the pop industry. The global artificial intelligence market size in music was valued at US$5.2 billion in 2024 and is projected to grow to approximately US$60.44 billion by 2034, growing at a compound annual growth rate (CAGR) of 27.8% [1]. The generative artificial intelligence segment is the fastest-growing subsector [2]. This rapid growth clearly demonstrates the interest in using machine learning in the music industry, and the programmed music business has transformed from an experimental field into a promising and rapidly growing field.

Artificial intelligence performs cognitive functions, namely learning and problem-solving. AI music is divided into two approaches: sound generation, where neural networks create raw sound waves (WAV), and symbolic generation, where algorithms process information in the form of sequences of notes (MIDI).

The history of programmatic music dates back to 1951, when Alan Turing, using a machine that occupied an entire floor of the Manchester Computer Laboratory, generated the first computer-generated musical composition. Today, anyone can generate a symphony in seconds. However, this ability is limited. The relevance of my project, which uses adaptive symbolic music generation, is that it can be competitive in the market by offering advantages that other market leaders in this field lack. Most commercial AI tools, such as Suno and Udio, create audio based on user input. If the user doesn't like certain instruments, notes, or chords in the melody, there's no way to change them except by generating a completely new melody.[3]

An important point is that music has the ability to positively impact people's emotional states. A research reveals that music therapy begins with music that matches the patient's current mood (even if it's depressive or anxious). Then the music has to be slowly changed to the desired mood. [4] Research also shows that music (particularly at a tempo of 60-80 bpm, which matches the heart rate) can reduce cortisol levels. [5] AI-powered music generation listening to user preferences can help to achieve these goals and positively impact mental well-being.

Instead of just generating audio, my project focuses on musical information in MIDI format and adapts to the user's preferences in real time. It listens to the user's reaction to the generated content. The user becomes a "teacher" and the program is a "student." MIDI format is useful for changes in BPM, exports to DAW, WAV, and musical notation. This approach will maximize the use of AI in music generation with a potentially positive impact on the user's emotional state.

I have defined the tasks necessary to succeed in the project. The first task is to analyze competitors for 2026 and identify their pros and cons to consider when implementing my solution.

The second task is to study music theory and reinforcement learning. There, I have to focus on how to interpret the user’s mood.
The third task is to analyze programming languages and libraries to ensure the most effective implementation of my ideas.
The fourth task is planning the project architecture using various diagrams.
The fifth task is coding.
Then, the project must evaluate its product and its potential impact on musicians and people interested in music.

The project's primary goal is to create unique and user-friendly AI music generation application that can inspire musicians to create new musical compositions and can be used in music therapy by ordinary users.

Research section

The first task. Analysis of competitors and market.

To create a unique, competitive program, it is necessary to research potential competitors. In 2026, AI in the music industry is rapidly gaining popularity, leading to the emergence of entertainment programs for the masses and professional tools for specialists.

Endel, a German competitor in the field of AI music for well-being, uses circadian rhythms, weather, and heart rate to generate nonstop soundscapes. It focuses on user relaxation, which is why it generates ambient music (a very slow, calm genre of music consisting primarily of keyboard instruments and synthesizer pads). However, it lacks a clear melody (musical notation), defined and changeable instruments, and is only an audio stream without the ability to export. Brain.FM, an American analogue, is similar in design, but claims to use a patented "Neural Phase Locking" technology. The music is specifically written (partly by AI, partly by humans) to modulate brainwaves. A competitor creates not only calm music for sleep and meditation, but also music tailored to the user's mood. It can change the energy, beat, and rhythm, and is used as background music for work and study, being particularly effective for people with ADHD. While both projects generate only calm or focusing music, my project's advantage lies in its structured music with specific instruments and notes, distinguished by its musicality. The user has control over the tempo, mood, and instruments, and can export to MIDI, WAV, and musical notation. My project also features a visualizer that displays key presses, which is especially useful for musicians.

A table of competitors in AI-based music generation for user well-being has been compiled.

My project is also aimed at composers, providing an excellent tool for inspiration. The Luxembourg-based AIVA project is a powerful AI composer for symphonic and electronic music, ideal for game and film soundtracks, with MP3 and MIDI export options. However, it focuses on production, presenting itself as a DAW platform that can be difficult to understand for the average user. The program isn't designed for individual preferences, nor does it build a musical profile. Generated tracks are licensed only with a subscription. An international competitor, Mubert, is a generative streaming service where users can generate compositions ranging from 5 seconds to 25 minutes long based on a prompt, genre selection, mood, and activity. The program offers an audio-only format, preventing users from selecting instruments, rhythm, or providing notation or MIDI files, which again is a drawback due to the lack of control over the composition. Orb Composer is a set of plugins built into a DAW that generate chord progressions and melodies based on user parameters (such as intensity). This project isn't designed to work individually with the user and won't remember their preferences.

Analysis revealed a unique advantage of my project. First, it has a musical structure. While competitors focused on therapy generate only abstract musical landscapes, my program creates a composition consisting of bass, chords, leads, and drums. Some people may find monotonous sounds uninteresting, requiring greater musicality. Second, my project supports MIDI export and musical notation. My program allows the user to change instruments and tempo on the fly, as well as export to musical notation and MIDI for further compositional work in a DAW, which is especially useful for professional composers. Third, the program creates a unique musical profile for the user, which the program then uses to learn and create music tailored to the user's preferences.

A comparison table of rivals and my project was created.

CRITERIAS	MY PROJECT	ENDEL	BRAIN.FM	AIVA	MUBERT	ORB COMPOSER
PRIMARY GOAL	**Music therapy and creativity**	Relaxation, sleep, focus	Brainwave modulation (for focus)	Professional soundtrack	Generative streaming, for background music	Assistance in DAW
MUSIC TYPE	**Structured composition (bass, chords, lead, drums)**	Endless soundscapes (ambient, noise)	Specially written music	Symphonic, electronic	Electronic music	Chord progressions & melodies
OUTPUT	**MIDI, WAV, musical notation**	Only audio stream	Only audio stream	MP3, MIDI	Audio only	MIDI (Internal DAW)
LEVEL OF CONTROL	**Change instruments, BPM, mood.**	Only listening	Only listening	Complex, professional software settings	Select genre/duration	Technical by parameter
PERSONALIZATION	**Unique user music profile.**	Biorhythms, weather, heart rate	Task-based (focus, relax)	None (does not learn from taste)	None (generates by prompt)	None (tool without memory)
VISUALIZATION	**Visualizes key presses and music profile**	Abstract animation	Abstract animation	None	Abstract animation	None (plugin GUI)
ACCESSIBILITY	**Universal. For Beginners, pros**	Mass market	Mass market	Professionals	Mass market	Professional composers

Table 1. Comparison of my project and its rivals.

The second task. Theory and methodology.

To understand music theory, we need to define what music is. It is the organization of sound in time, governed by pitch, rhythm, and harmony. The main element of music is the note, which is a sound of a particular pitch and duration. It refers to how low or high a sound is, and is represented using letters A through G (C-D-E-F-G-A-B – each letter is a different frequency. Notes are arranged in scales. Scales are sequences of notes in a particular order. Each scale is defined by intervals between notes measured in semitones. For instance, a major scale's pattern:

whole, whole, half, whole, whole, whole, half steps. The most widely used scales are major and minor. A major scale usually sounds happy, whileas minor scale sounds mournful.
A melody is a sequence of notes. The lead melody is usually sung by a vocalist. It can move step by step, which are adjacent notes in a scale, or by leaps, that are larger intervals. Stepwise motion is smoother, while as leaps cause tension or emphasis. The rhythm of a melody is the timing ad duration of notes, which create music's energy and motion. Notes are grouped into measures, which provide the structure to establish the meter, or the recurring pattern of strong and weak beats. The most common meter is 4/4 (4 beats per measure). Tempo, measured in beats per minute (BPM), determines how fast the music plays.
Harmony is the essential component of music. When notes are played simultaneously, they create chords which are built by stacking intervals of thirds on top of a root note. There are different chords: major, minor, diminished, or augmented chords. There is a chord progression – the harmonious sequence of chords in music. It gives the music a direction and a resolution. A common chord progression in Western music for major is "I-V-vi-IV", for minor is "i-VI-III-VII". Melody and harmony are influenced by scale degrees. For example, in a C major scale, C is the tonic, which serves as the tonal center, and the fifth note "G" provides a sense of tension and resolves back to the tonic.
Musical texture is also crucial for melody. There is a monophonic texture which consists of a single melody line, or homophonic textures, where a primary lead melody is supported by accompaniment (which are usually chords), which is popular in Western music. Bass line outlines the harmonic structure, and percussion instruments emphasize beats.
Dynamics and expression influence how music is perceived. They define the volume (loud or soft), how notes are played (short or smooth). Melody also includes tempo and harmony fluctuations to create a unique sound. By blending melody, harmony, rhythm and dynamics, the musical masterpiece is created. So, the music theory provides the basis for creating coherent and structured musical compositions.

Reinforcement learning was used for my project. The system will adapt to the user's preferences over time. RL is a section of AI where the model is trained to make decisions based on user feedback. In my program, the user is the "environment," and the generated track is the action. When the user likes or dislikes a track, the system will update the user profile to better match their preferences in the future. Unlike traditional supervised learning, where the model requires massive amounts of data, RL is adaptive, dynamic, and data-efficient. Since everyone has their own musical tastes, and these tastes can change, RL studies these changes, creating a personalized user profile. My project must learn based on data, user decisions, and behavior. The AI analyzes characteristics, applies mathematical models to melodies, and updates its knowledge about the user. The system learns from user feedback.

To use user mood, we need to define what constitutes "mood" for the computer, specifically, numerical values.
My program will use a vector of five different dimensions based on the Big Five personality traits (OCEAN). A demonstration of how this approach will be used is shown in the diagram

below.



Diagram 1. Five dimensions of a mood vector.

The adaptation of music profile in the project is influenced by simplified Reinforcement Learning Model. In classical RL, an “agent” takes an “action” in an “environment” to maximize a “reward”. For example, the agein is the “UserProfile” class, the action is generating a MIDI file with specific vectors, and the reward is user’s rating.
The main mathematical basis of the music profile update logic is the use of a weighted average with a momentum, which is calculated relative to the difference between the track and the profile. When a user rates a track, the system calculates the difference between the generated track's parameters and the user's current profile. If the rating is positive, the profile vectors shift toward the track's vectors. If the rating is negative, the profile vectors shift away from the track's vectors. The system constantly searches for the peak of user satisfaction in a five-dimensional space. A specific user interaction history with the program is a convenient and fast method,

unlike deep learning with huge datasets. This ensures rapid personalization, which is highly effective. A simplified explanation of how the vectors work in my project as a user reacts to a generated track is shown in the diagram below.

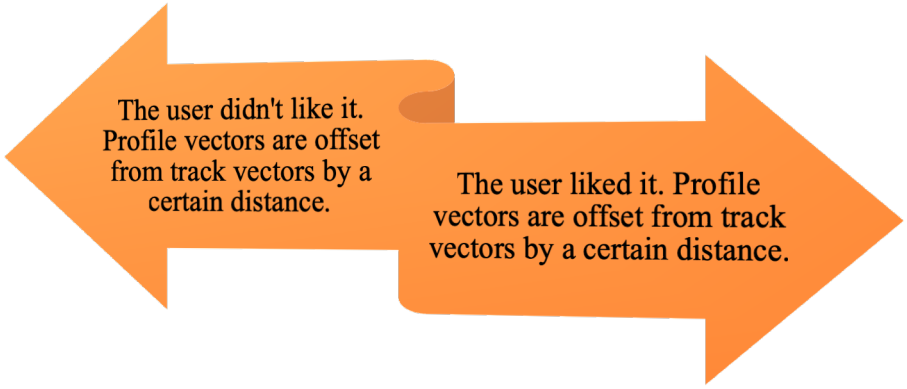


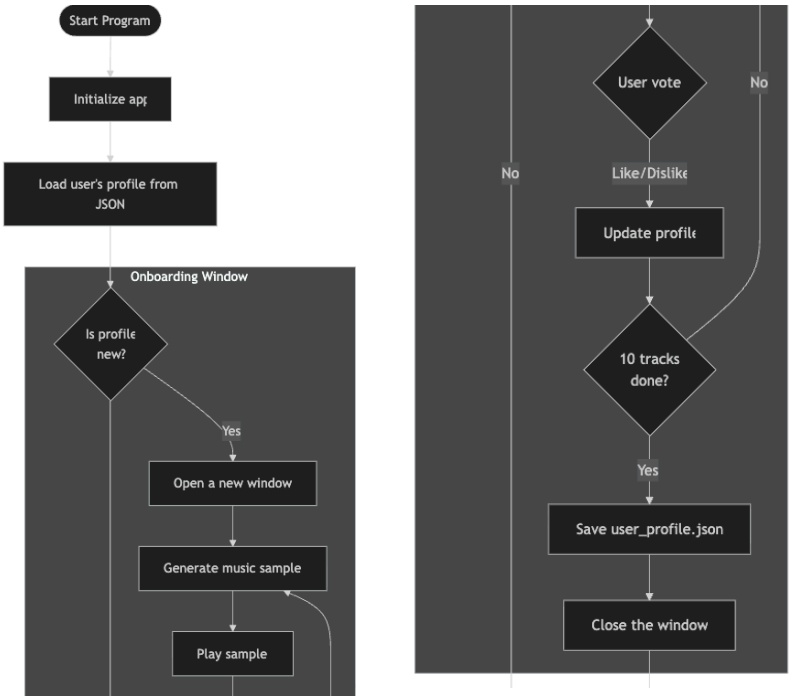
Diagram 2. Updating music profile.

Task 3. Programming language and libraries.
In this project, Python was chosen for numerous reasons.
According to the 2025 Stack Overflow Developer Survey, Python's popularity has increased, reaching nearly 58% in the data science and AI fields. This is a 7% increase from 2024, explaining its status as the "language of artificial intelligence" in the modern world. Python was chosen over C++ or Java for several reasons.

First, Python offers a huge number of specialized libraries that are perfectly suited for my project (GUI, mathematics, music theory).
Second, Python offers rapid prototyping. The interpreter allows for quick logic testing without lengthy compilation times.
Third, Python is very easy to work with lists and dictionaries. This makes it ideal for processing logic, MIDI, and data structures.
Fourth, although other languages have interface implementations, Python, given all the previous advantages, will be much easier.
C++ is too complex and requires manual memory management. One mistake will corrupt the entire code. Java is becoming obsolete in the field of artificial intelligence and lacks sufficient libraries for working with music and notation. It's difficult to implement complex AI training logic in JavaScript, making the project architecture too complex. Ultimately, Python proves to be the easiest and most optimal programming language.
Importantly, Python has specific libraries that other languages lack but are crucial for project creation.
Music21 is one of the most important libraries. It's a toolkit for computer musicology developed at MIT. Implementing music theory is easier because this library understands notes, chords, and other musical concepts. Artificial intelligence can think at the level of music theory rather than numerically, which is necessary for calculating depth and ensuring melody harmony. It can handle MIDI instructions like “Play middle C for 0.5 seconds”.
MIDIUtil is required for recording MIDI files. It accepts generated notes and efficiently converts them to MID format.

Tkinter is needed for the GUI. Unlike PyQt, which actually offers a more modern aesthetic, it is complex and adds additional costs. Tkinter is Python's standard library, ensuring program stability and fewer dependencies for running on different operating systems.
NumPy: UserProfile customization is based on vector mathematics (array addition, multiplication by a scalar). This library implements such complex operations, thereby ensuring high performance (instant profile updates).
Pygame: its role is to play the generated MIDI file and control the audio stream. For example, it can start, stop, or play continuously.
Matplotlib: required for plotting the Radar Chart (graphics) of the user profile, where the user can see their own profile vectors and track vectors. This ensures transparency of generation.
Also, additional libraries and system utilities were used: **PIL** for working with images, threading, which later turned out to be necessary for implementing export to wav (so that rendering occurs in the background without stopping the program), **subprocess** for running external programs (Musescore 4 for exporting and opening musical notation and FluidSynth for converting MIDI to wav via Soundfont) and **json** for storing user profile data.

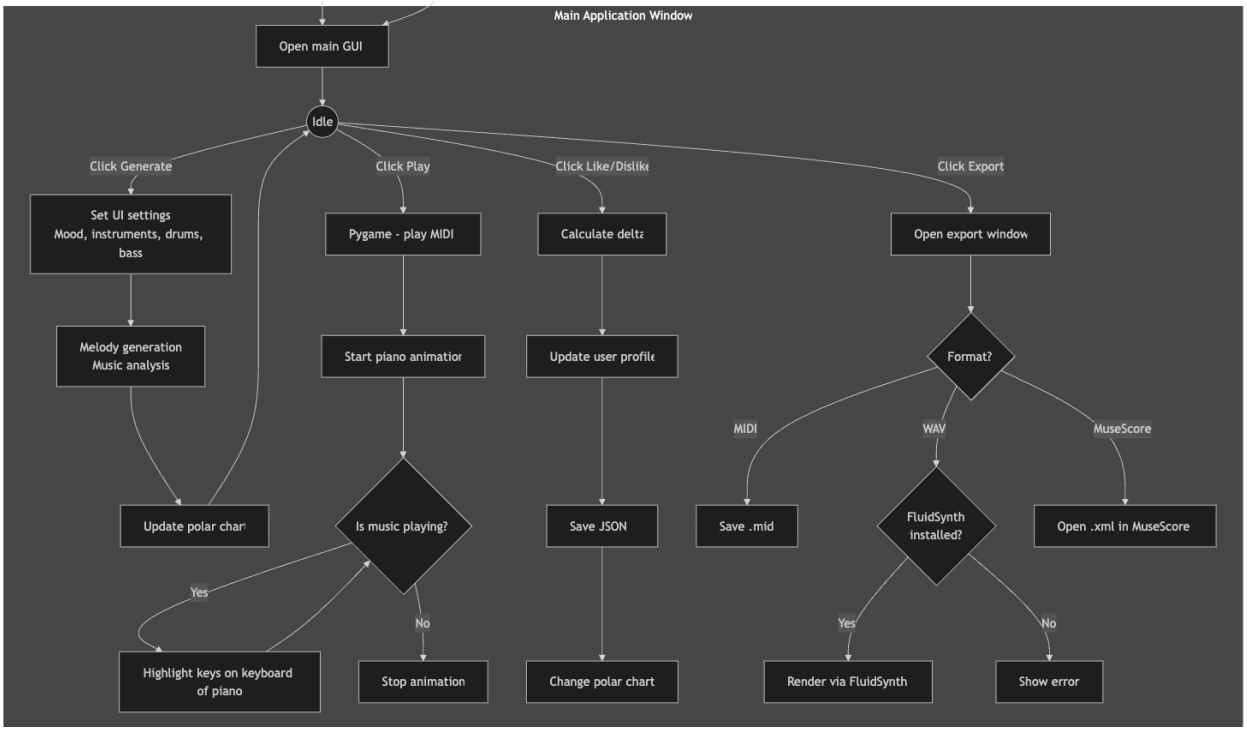
Task 4. System architecture.
The flowchart of starting program before setting main GUI was created (Flowchart 1) In case if an user opens the program at the first time, and there is no JSON file containing their profile, an user goes through the test with 10 different tracks. They can answer whether they like it or dislike it, and the program shapes their music profile. The test is implemented in a different, onboarding window. After this, JSON file of user's profile (containing vectors' values) is saved in the same folder of the main code. If user already has a music profile (JSON), the program will skip the test.



Flowchart 1. Initial user's test for music profile.

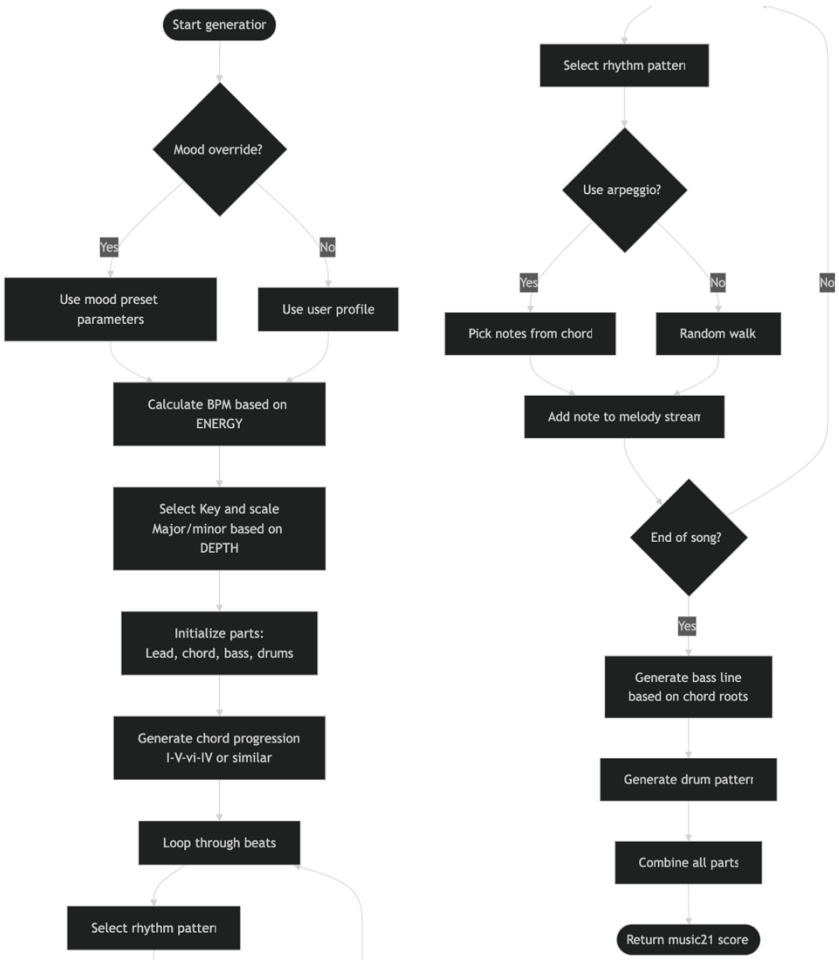
After test, main GUI opens. What concisely happens there is on the Flowchart 2. When a person click button "Generate", melody is generated and music is analyzed to understand the vectors of

the track and update polar chart for tracks. If Play button is clicked, pygame plays MIDI, using music21, and start piano animation, highlighting keys on piano keyboard (it is a melody visualization on piano). When user rates the generated melody by clicking Like or Dislike, their user profile is updated, changes are saved in the JSON file, and polar chart is updated. The system calculates the difference between the generated track's features and the stored profile. The profile is mathematically adjusted towards (if liked) or away from (if disliked) the track. When exporting, user has to choose the format – MIDI, WAV (audio), Musescore (for musical notation). If user asks to export in WAV, FluidSynth should be installed on the computer. If it is not installed, there is an error message.



Flowchart 2. The program's concise architecture.

To briefly understand how the music will be generated, Flowchart 3 was created. If mood is already chosen, the track is built by mood preset parameters. On the other hand, if user wants to create a track by their user profile, its vectors will be used. Then, BPM is calculated based on "Energy" vector's value, and key and scale are defined by "Depth" vector. To reduce the repetitiveness, controlled amount of random noise (e.g. 0.5) is added or subtracted from the vector. It guarantees creativity. Thus, The abstract vectors are mapped to concrete MIDI parameters (e.g., energy 0.8 means BPM 110, density of fast 1/16th notes). Parts of the song (chords, lead, drums, bass) are initialized, and chord progressions are generated. Subsequently, lead melody is generated: track may use arpeggio (when lead notes are similar to chord notes) or use random walk (random notes for key scale). All these are combined into a track, and recycled by music21 library functions to create a harmonious melody that can be played.



Flowchart 3. Melody generation.

Regarding architecture, the code is structured into three primary components.
Model (the brain) handles logic, data processing, music generation, mathematical analysis.
View handles the visual interpretation (piano keys) and receives user input.
Controller connects the user's interaction to the Model's logic and updates the View.

Task 5. Programming code.

To start a code, we need to import the libraries used.
From tkinter – tk, message box, filedialog.
From matplotlib – pyplot, FigureCanvasTkAgg
From music21 – stream, note, chord, tempo, key, roman, instrument, environment, pitch
From PIL – Image, ImageTk
Numpy, random, os, subprocess, platform, json, pygame, threading are imported.

```
SOUNDFONT_PATH = "soundfont.sf2"

us = environment.UserSettings()
us['musicxmlPath'] = '/Applications/MuseScore 4.app/Contents/MacOS/mscore'
us['musescoreDirectPNGPath'] = '/Applications/MuseScore 4.app/Contents/MacOS/mscore'
```

Code fragment 1.
Soundfont path is initialized. It is needed for exporting into WAV, as the MIDI file has to be played on a particular soundfont (set of instruments.
Also, the path to the MuseScore 4 app is initialized. This app is the most popular free app for music notation.

```
INSTRUMENTS = {
    "Piano": instrument.Piano,
    "Electric Piano": instrument.ElectricPiano,
    "Organ": instrument.PipeOrgan,
    "Marimba": instrument.Marimba,
    "Xylophone": instrument.Xylophone,
    "Acoustic Guitar": instrument.AcousticGuitar,
    "Electric Guitar": instrument.ElectricGuitar,
    "Violin": instrument.Violin,
    "Violoncello": instrument.Violoncello,
    "Flute": instrument.Flute,
    "Clarinet": instrument.Clarinet,
    "Trumpet": instrument.Trumpet,
    "Saxophone": instrument.AltoSaxophone,
    "Choir": instrument.Choir,
    "Koto": instrument.Koto,
    "Vocalist": instrument.Vocalist,
    "Pad": instrument.Ukulele
}
```

```
INSTRUMENT_ICONS = {
    "Piano": "🎹",
    "Electric Piano": "🎹⚡",
    "Organ": "⛪",
    "Marimba": "🪵",
    "Xylophone": "🦴",
    "Acoustic Guitar": "🎸",
    "Electric Guitar": "🎸⚡",
    "Violin": "🎻",
    "Violoncello": "🎻⬇️",
    "Flute": "🪈",
    "Clarinet": "🎷⬛",
    "Trumpet": "🎺",
    "Saxophone": "🎷",
    "Choir": "🎹☁️",
    "Koto": "🇯🇵",
    "Vocalist": "🎤",
    "Pad": "🎛️"
}
```

Code fragment 2.

Thanks to music21, we can easily import a variety of instruments using an Instrument tool. I initialize them in a set. Additionally, icons of instruments are provided in an “Emoji” style, eliminating the need for additional icon images to be installed.

```
COLORS = {
    "bg_main": "#121212",
    "bg_panel": "#1E1E1E",
    "text_main": "#E0E0E0",
    "text_dim": "#888888",
    "highlight": "#3498DB",

    "shadow": "#080808",

    "btn_base": "#25303B",
    "btn_hover": "#34495E",

    "btn_green": "#1E8449",
    "btn_green_hover": "#27AE60",

    "btn_red": "#922B21",
    "btn_red_hover": "#C0392B",

    "btn_blue": "#1F618D",
    "btn_blue_hover": "#2980B9",
    "graph_bg": "#2B2B2B",
    "graph_line": "#3498DB",
    "text_success": "#58D68D",
    "text_error": "#EC7063"
}
```

Code fragment 3.
For the better appearance of GUI, a color palette was created in the set COLORS. It’ll be easier in the creation of GUI to call out these colors for text, boxes, background, etc.

```python
class ShadowButton(tk.Frame):
    def __init__(self, parent, text, command=None,
                 width=20, height=2,
                 bg_color=COLORS["btn_base"], fg_color=COLORS["text_main"],
                 hover_color=COLORS["btn_hover"],
                 shadow_color=COLORS["shadow"],
                 font=("Arial", 11, "bold")):

        super().__init__(parent, bg=shadow_color)

        self.command = command
        self.bg_color = bg_color
        self.hover_color = hover_color
        self.fg_color = fg_color
        self.is_disabled = False

        self.lbl = tk.Label(self, text=text, bg=bg_color, fg=fg_color, font=font,
                            width=width, height=height, cursor="hand2")

        self.lbl.pack(fill=tk.BOTH, expand=True, padx=(0, 3), pady=(0, 3))

        self.lbl.bind("<Button-1>", self.on_click)
        self.lbl.bind("<Enter>", self.on_enter)
        self.lbl.bind("<Leave>", self.on_leave)

    def on_click(self, event):
        if self.command and not self.is_disabled:
            self.lbl.pack_configure(padx=(1, 0), pady=(1, 0))
            self.update_idletasks()
            self.after(100, lambda: self.lbl.pack_configure(padx=(0, 3), pady=(0, 3)))
            self.command()

    def on_enter(self, event):
        if not self.is_disabled: self.lbl.configure(bg=self.hover_color)
    def on_leave(self, event):
        if not self.is_disabled: self.lbl.configure(bg=self.bg_color)
    def config_state(self, state="normal"):
        if state == "disabled":
            self.is_disabled = True
            self.lbl.configure(fg="#555", cursor="arrow")
        else:
            self.is_disabled = False
            self.lbl.configure(fg=self.fg_color, cursor="hand2")
```

Code fragment 4.

The class ShadowButton was created for aesthetically pleasing UI. It is a custom Tkinter button widget that looks like a button with a shadow, reacts to hover, animates a press effect, and it is built from a tk.Frame (shadow layer) and a tk.Label (visible button surface). The frame acts as the shadow container and a background. The label inside is an actual button. Using (0, 3, 3) padding, a label is slightly shifted left, revealing the shadow behind it. When a user clicks it, a label goes down-right to look pressed. Then after time, it returns to its initial position. When a mouse enters the button, background changes (hover effect).

```python
class MusicAnalyst:
    @staticmethod
    def analyze_stream(music_stream):
        features = {}

        try:
            melody_part = music_stream.parts[0] if music_stream.parts else music_stream
        except:
            melody_part = music_stream

        flat_notes = melody_part.flatten().notes

        #energy
        bpm = 120
        try:
            mm = music_stream.metronomeMarkBoundaries()
            if mm: bpm = mm[0][2].number
        except: pass

        norm_bpm = min(max((bpm - 60) / 120, 0.0), 1.0)

        if len(flat_notes) > 0:
            total_duration = music_stream.duration.quarterLength
            if total_duration == 0: total_duration = 1
            density = len(flat_notes) / total_duration
            norm_density = min(density / 2.0, 1.0)
        else:
            norm_density = 0.5

        features['energy'] = (norm_bpm * 0.6) + (norm_density * 0.4)

        #depth
        pitches = [n.pitch.midi for n in flat_notes if n.isNote]
        if len(pitches) >= 2:
            pitch_range = max(pitches) - min(pitches)
            mean_pitch = sum(pitches) / len(pitches)
            range_score = min(pitch_range / 24.0, 1.0)
            center_distance = abs(mean_pitch - 60)
            center_score = max(1.0 - center_distance / 24.0, 0.0)
            intervals = [abs(pitches[i] - pitches[i-1]) for i in range(1, len(pitches))]
            avg_interval = sum(intervals) / len(intervals)
            interval_score = min(avg_interval / 7.0, 1.0)

            features['depth'] = min(0.4 * range_score + 0.4 * center_score + 0.2 * interval_score, 1.0)
        else:
            features['depth'] = 0.3
```

```python
        #openness
        melody_pitches = [p.ps for p in melody_part.flatten().pitches]
        if melody_pitches:
            p_range = max(melody_pitches) - min(melody_pitches)
            features['openness'] = min(p_range / 48.0, 1.0)
        else:
            features['openness'] = 0.5

        #complexity
        intervals = []
        if len(pitches) > 1:
            for i in range(len(pitches)-1):
                intervals.append(abs(pitches[i+1] - pitches[i]))
            avg_interval = sum(intervals)/len(intervals) if intervals else 0
            features['complexity'] = min(avg_interval / 7.0, 1.0)
        else:
            features['complexity'] = 0.0

        #structure
        from collections import Counter
        durations = [round(n.duration.quarterLength, 2) for n in flat_notes]
        if durations:
            counts = Counter(durations)
            most_common_ratio = counts.most_common(1)[0][1] / len(durations)
            features['structure'] = min(max(most_common_ratio * 1.4, 0.0), 1.0)
        else:
            features['structure'] = 0.5

        return features

    @staticmethod
    def profile_distance(track_features, user_profile):
        keys = ['energy', 'depth', 'complexity', 'openness', 'structure']
        diffs = []
        for k in keys:
            diffs.append(abs(track_features.get(k, 0.5) - user_profile.get(k, 0.5)))
        return sum(diffs) / len(diffs)
```

Code fragments 5 and 6.

I created a MusicAnalysist class. It takes a music21 stream (a parsed MIDI/score), extracts numerical musical characteristics in the range 0.0-1.0, and compares those characteristics to a user music profile and returns a distance score. That is, we say to it “Convert music into a 5-dimensional personality vector, then measure how similar it is to a user’s taste”.
Def “analyse_stream” analyzes a music piece and returns a set of vectors with their values ({‘energy’: float, ‘depth’: float, …}). Initially, if the stream has multiple parts, the def takes the first part (assumed melody), otherwise it treats the whole stream as melody. Then, with .flatten(), it removes measures and voices, giving a linear sequence of notes.
Energy is a mix of tempo and note density. BPM is initialized as 120 by default; if tempo exists, a program uses it. Norm_bpm normalizes bpm. Regarding density, more notes per beat mean higher energy, whereas sparse music means lower energy. To find it, the code counts the total number of notes and divides by the duration in second. Density is calculated and normalized. Then, the final energy is found by a formula, where tempo is weighted 10% more than note count. Since BPM can range from 60 to 200 and Density from 1 to 20 notes/sec, the code normalizes these values to a 0.0 to 1.0 scale.
Depth tries to measure emotional richness. It considers pitch range, where wider range means more depth. Range_score is divided by 24 semitons (2 octaves). Center_distance: 60 is middle C; if melodies are centered near middle, they are more balanced; if it is too high or too low, there is less depth. Interval movement: if there are larger melodic jumps, the music is expressive; if a melody is stepwise, it is flatter emotionally. By the measurements (where the standard deviation of the pitch distribution is found), a final depth score is counted and clamped to 1.0.
Openness is how wide the melodic space is. That is, how many semitones are used. A program finds the range and divides it by 48 semitones. Very wide melodies are open, whereas narrow melodies are closed and intimate, resulting in less openness.
Complexity measures how predictable the melody is. A scale (C, D, E, F) is predictable; a random sequence is not. It is based purely on average interval size. Intervals between notes (for example, from C2 to C3 is a big interval, while from C2 to D2 is not) are counted and interpreted by a program to create the vector of complexity. Large jumps mean complex, while mostly stepwise melody means “simple.” This is melodic complexity, not rhythmic or harmonic.
Structure measures rhythmic regularity. The program finds note durations in a melody and deciphers most common ratio by diving most common duration count by total notes. Then, the ratio is multiplied by 1.4 to make it scaled. Basically, if the same rhythm is repeated, structure is higher; if there are many different note lengths, structure is lower.
Then, the def “profile_distance” measures how close a track is to a user’s taste by. Finding average absolute difference. That is, the difference for each dimension is found (for instance, track_value – user_value). 0.0 means perfect match, while 1.0 counts as a complete mismatch.

```python
class UserProfile:
    def __init__(self):
        base_dir = os.path.dirname(os.path.abspath(__file__))
        self.filename = os.path.join(base_dir, "user_profile.json")

        self.learning_rate = 0.4
        self.dislike_factor = 0.7
        self.profile = {
            'energy': 0.5, 'depth': 0.5, 'complexity': 0.5,
            'openness': 0.5, 'structure': 0.5
        }
        self.load_profile()

    def update_profile(self, track_features, rating):
        for key in self.profile:
            current_val = self.profile[key]
            track_val = track_features.get(key, 0.5)
            diff = track_val - current_val
            if rating > 0:
                delta = diff * self.learning_rate
            else:
                delta = -diff * self.learning_rate * self.dislike_factor

            delta += 0.2 * (1 if delta > 0 else -1) * abs(diff)
            new_val = current_val + delta
            self.profile[key] = max(0.01, min(new_val, 0.99))

        self.save_profile()
        print(f"[Profile updated] {self.profile}")

    def save_profile(self):
        try:
            with open(self.filename, 'w') as f:
                json.dump(self.profile, f, indent=4)
            print(f"--> JSON saved: {self.filename}")
        except Exception as e:
            print(f"SAVE ERROR: {e}")
            try:
                messagebox.showerror("ERROR", f"Profile could not be saved:\n{e}")
            except: pass

    def load_profile(self):
        if os.path.exists(self.filename):
            try:
                with open(self.filename, 'r') as f:
                    self.profile = json.load(f)
            except:
                pass
        else:
            print("New profile is created.")

    def get_vector(self):
        return [self.profile['energy'], self.profile['depth'], self.profile['complexity'],
                self.profile['openness'], self.profile['structure']]
```

Code fragment 7.

A UserProfile class was created. It represents a learning preference model for a user's musical taste. Its job is to store the user's taste as a 5-dimensional vector, update that taste based on feedback, persist the profile to disk to JSON, and provide the profile in a format usable by other systems (e.g. MusicAnalysist). This is a reinforcement-style preference learner, not machine learning. Def "init" ensures the profile file is stored next to the script and makes paths portable across systems. The learning_rate variable means how fast prefences change, and the dislike_factor shows how dislike influences the profile less strongly than likes. Initial neutral profile"s vectors are 0.5, meaning they are neutral in every dimension. If a profile exists, "load_profile()" overwrites defaults. Def "update_profile" is the core learning logic. Its inputs are track_features from MusicAnalysist and rating (like or dislike). Then, the function computes the difference: if difference is bigger than zero, a track has more of this trait than the user, and vice versa. If user likes the track, the profile is moved toward the track; if user dislikes the track, it pushes the profile away from the track. Delta is changed to be prepared for directional bias. It prevents tiny differences from habing no effect. In new_val and self.profile[key], values are updated and clamped to prevent exact 0 or 1, and avoid dead-ends. The profile is immediately saved. Def "load_profile()" loads the profile if it exists, def "save_profile()" saves the profile to JSON, handling errors (the errors are printed to console error). Def "get_vector()" converts the dictionary into a fixed-order vector that is ideal for distance calculations, matching algorithms, and optimization logic. It perfectly aligns with Music.Analyst.profile_distance. The class ensures online learning and gradient-like preference adaptation.

```
class MelodyGenerator:
    def inject_creativity(self, profile, amount=0.08):
        creative = profile.copy()
        for k in creative:
            delta = random.uniform(-amount, amount)
            creative[k] = max(0.01, min(creative[k] + delta, 0.99))
        return creative

    def corrected_profile(self, base_profile, track_features, strength=0.5):
        corrected = base_profile.copy()
        for k in base_profile:
            tf = track_features.get(k)
            if tf is None: continue
            corrected[k] = base_profile[k] + (base_profile[k] - tf) * strength
            corrected[k] = max(0.01, min(corrected[k], 0.99))
        return corrected

    def resolve_instruments(self, name):
        instr = INSTRUMENTS.get(name, instrument.Piano)
        return instr(), instrument.Piano()

    def resolve_lead_chord_instruments(self, lead_name, chord_name):
        lead_instr = INSTRUMENTS.get(lead_name, instrument.Piano)()
        chord_instr = INSTRUMENTS.get(chord_name, instrument.Piano)()
        return lead_instr, chord_instr
def generate_track(self, profile, mood_override=None,
                   lead_instrument="Piano",
                   chord_instrument="Piano"):

    is_strict_mode = (mood_override is None)
    p = profile.copy()

    if not is_strict_mode:
        p.update(mood_override)
        def soft_clamp(value, target, strength=0.92, noise=0.0):
            val = target * strength + value * (1 - strength)
            val += random.uniform(-noise, noise)
            return max(0.01, min(val, 0.99))

        track_features = p.copy()
        for k in ['energy', 'depth', 'complexity', 'openness', 'structure']:
            track_features[k] = soft_clamp(track_features.get(k, 0.5), p[k], strength=0.85, noise=0.07)
        p = track_features

    bpm = int(60 + p['energy'] * 120)
    mode = 'minor' if p['depth'] > 0.55 else 'major'

    melody_instr, harmony_instr = self.resolve_lead_chord_instruments(lead_instrument, chord_instrument)

    root_notes = ['C', 'D', 'E', 'F', 'G', 'A', 'B']
    root = random.choice(root_notes)
    melody_key = key.Key(root, mode=mode)
    harmony_key = key.Key(root, mode=mode)

    scale_pitches = [pitch.midi for pitch in melody_key.getPitches("G3", "C6")]
    center_idx = len(scale_pitches) // 2
    range_width = int(len(scale_pitches) * (0.2 + 0.6 * p['openness']))
    min_idx = max(0, center_idx - range_width // 2)
    max_idx = min(len(scale_pitches) - 1, center_idx + range_width // 2)

    s = stream.Score()
    part_melody = stream.Part()
    part_harmony = stream.Part()
    part_drums = stream.Part()
    part_bass = stream.Part()

    part_melody.append(tempo.MetronomeMark(number=bpm))
    part_melody.insert(0, melody_instr)
    part_harmony.insert(0, harmony_instr)

    # drums on the 10ch
    perc_inst = instrument.Percussion()
    try: perc_inst.midiChannel = 9
    except: pass
    part_drums.insert(0, perc_inst)
#bass
bass_inst = instrument.ElectricBass()
part_bass.insert(0, bass_inst)

part_melody.append(melody_key)
part_harmony.append(harmony_key)

base_prog = ['I', 'V', 'vi', 'IV'] if mode == 'major' else ['i', 'VI', 'III', 'VII']
progression = base_prog * 8

chord_objs = []
for rom in progression:
    rn = roman.RomanNumeral(rom, harmony_key)
    c = chord.Chord(rn.pitches)
    c.quarterLength = 4.0
    c.closedPosition(forceOctave=3, inPlace=True)
    part_harmony.append(c)
    chord_objs.append(c)

#melody
curr_index = center_idx
best_dist = 999
for i in range(min_idx, max_idx+1):
    if scale_pitches[i] % 12 == melody_key.tonic.midi % 12:
        if abs(i - center_idx) < best_dist:
            best_dist = abs(i - center_idx)
            curr_index = i

total_beats = len(progression) * 4
current_beat = 0
rhythm_patterns = [[1.0, 1.0], [0.5, 1.0, 0.5], [2.0], [1.5, 0.5], [0.5, 0.5, 1.0], [1.0, 0.5, 0.5]]

if is_strict_mode:
    if p['complexity'] < 0.25: allowed_steps = [1, -1, 1, -1, 2, -2]
    elif p['complexity'] < 0.6: allowed_steps = [0, 1, -1, 1, -1, 2, -2, 3, -3]
    else: allowed_steps = [1, -1, 2, -2, 3, -3, 4, -4, 0]
else:
    allowed_steps = [0, 1, -1, 1, -1, 2, -2, 3, -3]

arp_patterns = [[0, 1, 2, 1], [0, 2, 1, 0], [0, 1, 2, 3]]
current_rhythm = random.choice(rhythm_patterns)
```

Code fragment 8.

MelodyGenerator is the core of the system that generates music. Its purpose is to create musical compositions based on a user profile. Def "inject_creativity" sets controlled randomness into a base profile. If the program strictly follows the user profile, the resulting music would become too repetitive. For each profile dimension, a random value in the range is added. The result is clamped between 0.01 and 0.99. The creativity remains subtle and musically plausible.

Def "corrected_profile(base_profile, track_features, strength=0.5)" intentionally pushes new music away from the analyzed track features. It avoids stagnation and encourages exploration. For each dimension, "corrected = base + (base − track) × strength" is applied.

Def "resolve_intruments" maps a user-friendly instrument name ("Piano") to a music21.Instrument.

Def "resolve_lead_chord_instruments(lead_name, chord_name)" resolves instruments for lead melody and chords accompaniment.

Def "generate_track()" is the central method of the class. It generates an entire melody as a music21.stream.Score. It operates in two modes: strict mode when mood_override is None (meaning that the music should be generated by the music profile); override mode (when a mood preset is applied).

"bpm = 60 + energy × 120" – energy calculates the BPM of the track.

"minor if depth > 0.55 else major" – depth correlates with the mode selection (major or minor).

The scale in the program spans from G3 to C6. Openness controls melodic range.

Then, the generator creates four independent parts: melody, harmony (chords), drums, and bass. Each part is synchronized.

Chord progression is selected for major and minor. For major, it is "I – V – vi – IV"; for minor, it is "I – VI – III – VII". These are harmonically stable and familiar to listeners.

```
while current_beat < total_beats:
    if is_strict_mode:
        if p['structure'] < 0.8 and random.random() < (1 - p['structure']):
            current_rhythm = random.choice(rhythm_patterns)
    else:
        if random.random() < 0.4: current_rhythm = random.choice(rhythm_patterns)

    rhythm = current_rhythm
    use_arp = False
    if p['complexity'] > 0.3 or (not is_strict_mode and random.random() < 0.3):
        use_arp = random.random() < 0.3
    arp = random.choice(arp_patterns) if use_arp else None

    for i in range(len(rhythm)):
        dur = rhythm[i]
        if current_beat + dur > total_beats:
            dur = total_beats - current_beat
            if dur <= 0: break

        chord_idx = int(current_beat // 4)
        chord_obj = chord_objs[chord_idx] if chord_idx < len(chord_objs) else None
        chord_tones_midi = [pit.midi % 12 for pit in chord_obj.pitches] if chord_obj else []

        if chord_tones_midi and arp:
            possible_indices = [idx for idx in range(min_idx, max_idx) if scale_pitches[idx] % 12 in chord_tones_midi]
            if possible_indices:
                closest_idx = min(possible_indices, key=lambda x: abs(x - curr_index))
                arp_offset = arp[i % len(arp)]
                try:
                    curr_pos_in_chord = 0
                    for k, p_idx in enumerate(possible_indices):
                        if p_idx == closest_idx: curr_pos_in_chord = k; break
                    next_index = possible_indices[(curr_pos_in_chord + arp_offset) % len(possible_indices)]
                except: next_index = closest_idx
            else: next_index = curr_index
        else:
            step = random.choice(allowed_steps)
            if is_strict_mode and p['complexity'] < 0.3 and step == 0: step = random.choice([-1, 1])
            next_index = curr_index + step

        next_index = max(min_idx, min(max_idx, next_index))
        candidate_pitch = scale_pitches[next_index]

        should_harmonize = False
        if chord_tones_midi and (candidate_pitch % 12) not in chord_tones_midi:
            if is_strict_mode or random.random() < p['structure']: should_harmonize = True
```

Code fragment 9.

The melody starts near the tonic pitch closest to the center of the pitch range. Predefined rhythmic templates are used for lead melody. Structure influences it: with high structure, rhythm patterns are reused; with low structure, there are frequent changes.

Allowed pitch steps depend on complexity: low complexity means stepwise motion, while high complexity means wide intervals and rests. At higher complexity levels, arpeggios are introduced. Notes are chosen from the current chord notes. It increases musical richness.

Also, harmony is constantly corrected. If a generated note does not belong to the current chord, the system searches nearby pitches and corrects toward chord tones to ensure consonance.

"velocity = 55 + energy × 55" – higher energy produces louder and faster melodies.

Bass lines are based on energy. Low energy means long sustained notes, while high energy means rhythmic bass patterns. Higher complexity results in syncopated bass figures. They are octave-adjusted to remain within a normal bass register.

Drums are generated algorithmically using MIDI percussion notes. Hi-hat on eighth notes, kick on strong beats, snare on beats 2 and 4, crash cymbal every 16 beats – it is a popular drum pattern.

All parts are appended to the score in the order of Melody, Harmony, Drums, and Bass. The final music21.stream.Score can be played back, exported as MIDI, rendered in WAV and exported in notation software.

```
    # drums sound like dis
    drum_duration = 0.5
    total_eighths = int(total_beats * 2)

    for i in range(total_eighths):
        beat_pos = i / 2.0
        notes_to_play = []
        is_crash_hit = (beat_pos % 16 == 0)

        if is_crash_hit:
            notes_to_play.append(49)
            notes_to_play.append(36)
        else:
            notes_to_play.append(42)

        pos_in_bar = beat_pos % 4.0
        if not is_crash_hit:
            if pos_in_bar == 0.0 or pos_in_bar == 2.0: notes_to_play.append(36)
        if pos_in_bar == 1.0 or pos_in_bar == 3.0: notes_to_play.append(38)

        if notes_to_play:
            notes_to_play = list(set(notes_to_play))
            if len(notes_to_play) > 1:
                c = chord.Chord(notes_to_play)
                c.quarterLength = drum_duration
                c.volume.velocity = 85
                part_drums.append(c)
            else:
                n = note.Note(notes_to_play[0])
                n.quarterLength = drum_duration
                n.volume.velocity = 85
                part_drums.append(n)
        else:
            r = note.Rest()
            r.quarterLength = drum_duration
            part_drums.append(r)

    s.append(part_melody)
    s.append(part_harmony)
    s.append(part_drums)
    s.append(part_bass)
    return s
```

```
            if should_harmonize and chord_tones_midi:
                search_offsets = [0, 1, -1, 2, -2]
                for off in search_offsets:
                    check_idx = next_index + off
                    if min_idx <= check_idx <= max_idx:
                        if (scale_pitches[check_idx] % 12) in chord_tones_midi:
                            next_index = check_idx; break

            curr_index = next_index
            final_pitch = scale_pitches[curr_index]

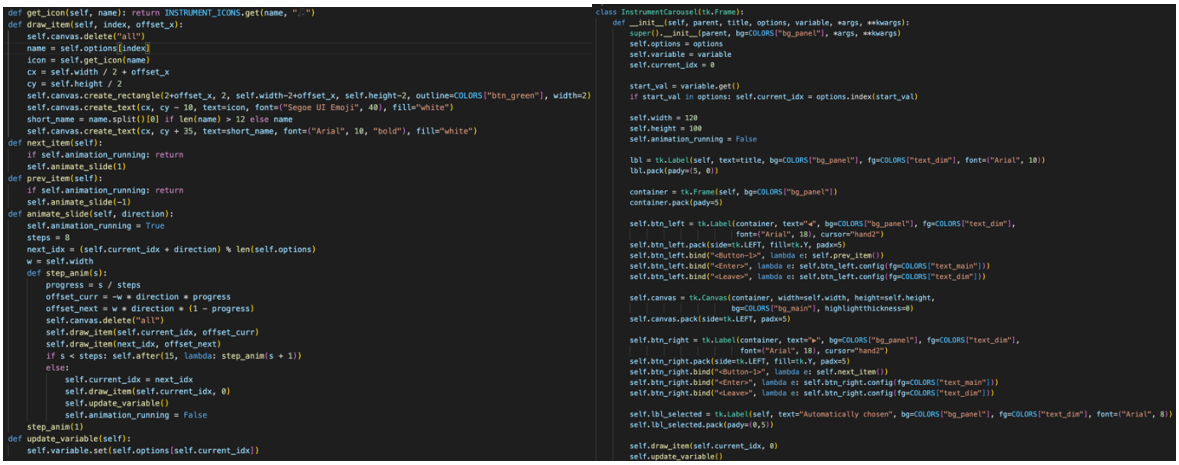
            n = note.Note(final_pitch)
            n.quarterLength = dur
            n.volume.velocity = int(55 + p['energy'] * 55)
            part_melody.append(n)
            current_beat += dur
            if current_beat >= total_beats: break

    # bass sound
    for i in range(len(progression)):
        c_obj = chord_objs[i]
        root_midi = c_obj.root().midi
        while root_midi > 52: root_midi -= 12
        while root_midi < 28: root_midi += 12

        # bass sounds like dis
        if p['energy'] > 0.6:
            pattern = [(1.0, True), (2.0, False), (1.0, True)]
            if p['complexity'] > 0.6:
                pattern = [(0.5, True), (0.5, True), (0.5, False), (0.5, True)] * 2
        else:
            pattern = [(4.0, True)]

        for dur, is_note in pattern:
            if is_note:
                n = note.Note(root_midi)
                n.quarterLength = dur
                n.volume.velocity = 90
                part_bass.append(n)
            else:
                r = note.Rest()
                r.quarterLength = dur
                part_bass.append(r)
```

Code fragment 10 and 11.



```
def get_icon(self, name): return INSTRUMENT_ICONS.get(name, "♪")
def draw_item(self, index, offset_x):
    self.canvas.delete("all")
    name = self.options[index]
    icon = self.get_icon(name)
    cx = self.width / 2 + offset_x
    cy = self.height / 2
    self.canvas.create_rectangle(2+offset_x, 2, self.width-2+offset_x, self.height-2, outline=COLORS["btn_green"], width=2)
    self.canvas.create_text(cx, cy - 10, text=icon, font=("Segoe UI Emoji", 40), fill="white")
    short_name = name.split()[0] if len(name) > 12 else name
    self.canvas.create_text(cx, cy + 35, text=short_name, font=("Arial", 10, "bold"), fill="white")
def next_item(self):
    if self.animation_running: return
    self.animate_slide(1)
def prev_item(self):
    if self.animation_running: return
    self.animate_slide(-1)
def animate_slide(self, direction):
    self.animation_running = True
    steps = 8
    next_idx = (self.current_idx + direction) % len(self.options)
    w = self.width
    def step_anim(s):
        progress = s / steps
        offset_curr = -w * direction * progress
        offset_next = w * direction * (1 - progress)
        self.canvas.delete("all")
        self.draw_item(self.current_idx, offset_curr)
        self.draw_item(next_idx, offset_next)
        if s < steps: self.after(15, lambda: step_anim(s + 1))
        else:
            self.current_idx = next_idx
            self.draw_item(self.current_idx, 0)
            self.update_variable()
            self.animation_running = False
    step_anim(1)
def update_variable(self):
    self.variable.set(self.options[self.current_idx])
```

```
class InstrumentCarousel(tk.Frame):
    def __init__(self, parent, title, options, variable, *args, **kwargs):
        super().__init__(parent, bg=COLORS["bg_panel"], *args, **kwargs)
        self.options = options
        self.variable = variable
        self.current_idx = 0

        start_val = variable.get()
        if start_val in options: self.current_idx = options.index(start_val)

        self.width = 120
        self.height = 100
        self.animation_running = False

        lbl = tk.Label(self, text=title, bg=COLORS["bg_panel"], fg=COLORS["text_dim"], font=("Arial", 10))
        lbl.pack(pady=(5, 0))

        container = tk.Frame(self, bg=COLORS["bg_panel"])
        container.pack(pady=5)

        self.btn_left = tk.Label(container, text="◄", bg=COLORS["bg_panel"], fg=COLORS["text_dim"],
                                 font=("Arial", 18), cursor="hand2")
        self.btn_left.pack(side=tk.LEFT, fill=tk.Y, padx=5)
        self.btn_left.bind("<Button-1>", lambda e: self.prev_item())
        self.btn_left.bind("<Enter>", lambda e: self.btn_left.config(fg=COLORS["text_main"]))
        self.btn_left.bind("<Leave>", lambda e: self.btn_left.config(fg=COLORS["text_dim"]))

        self.canvas = tk.Canvas(container, width=self.width, height=self.height,
                                bg=COLORS["bg_main"], highlightthickness=0)
        self.canvas.pack(side=tk.LEFT, padx=5)

        self.btn_right = tk.Label(container, text="►", bg=COLORS["bg_panel"], fg=COLORS["text_dim"],
                                  font=("Arial", 18), cursor="hand2")
        self.btn_right.pack(side=tk.LEFT, fill=tk.Y, padx=5)
        self.btn_right.bind("<Button-1>", lambda e: self.next_item())
        self.btn_right.bind("<Enter>", lambda e: self.btn_right.config(fg=COLORS["text_main"]))
        self.btn_right.bind("<Leave>", lambda e: self.btn_right.config(fg=COLORS["text_dim"]))

        self.lbl_selected = tk.Label(self, text="Automatically chosen", bg=COLORS["bg_panel"], fg=COLORS["text_dim"], font=("Arial", 8))
        self.lbl_selected.pack(pady=(0,5))

        self.draw_item(self.current_idx, 0)
        self.update_variable()
```

Code fragment 12.

For better GUI, the InstrumentCarousel class was created, It is a custom Tkinter widget that provides an animated instrument selector. Its purpose is to allow the user to cycle through a predefined list of instruments while keeping the selected instrument synchronized with the program. It inherits from tk.Frame and manages a list of available instruments, tk.Stringvar that stores currently selected instrument, and an index that tracks which option is available. When the user clicks an arrow, animate_slide(direction) is called. It consists of a title label, a left arrow and right arrow, a central Canvas where the instrument icon and name are drawn, and a small statur label showing the current selection The instrument in the carousel is changed with a sliding animation, current_idx is updated and the bound StringVar is set.



```
class PianoKeyboard(tk.Canvas):
    def __init__(self, parent, width=700, height=100, start_note=48, end_note=84):
        super().__init__(parent, width=width, height=height, bg="#333", highlightthickness=0)
        self.start_note = start_note
        self.end_note = end_note
        self.white_keys = []
        self.black_keys = []

        total_keys = 0
        for i in range(start_note, end_note + 1):
            if i % 12 in [0, 2, 4, 5, 7, 9, 11]: # white klavishi
                total_keys += 1

        self.key_w = width / total_keys
        self.key_h = height
        self.draw_keyboard()

        self.active_notes = set()

    def draw_keyboard(self):
        self.delete("all")
        self.key_map = {} # midi -> canvas_id

        x = 0
        # white keys
        for pitch in range(self.start_note, self.end_note + 1):
            note_in_octave = pitch % 12
            is_white = note_in_octave in [0, 2, 4, 5, 7, 9, 11]

            if is_white:
                rect = self.create_rectangle(x, 0, x + self.key_w, self.key_h,
                                             fill="white", outline="black", tags=f"key_{pitch}")
                self.key_map[pitch] = rect
                # C note text
                if note_in_octave == 0:
                    self.create_text(x + self.key_w/2, self.key_h - 15, text=f"C{pitch//12 - 1}", fill="#ccc", font=("Arial", 8))
                x += self.key_w
```

```
        # black keys
        x = 0
        for pitch in range(self.start_note, self.end_note + 1):
            note_in_octave = pitch % 12
            is_white = note_in_octave in [0, 2, 4, 5, 7, 9, 11]

            if is_white:
                x += self.key_w
            else:
                #black a lil left
                bx = x - (self.key_w * 0.35)
                rect = self.create_rectangle(bx, 0, bx + self.key_w * 0.7, self.key_h * 0.6,
                                             fill="black", outline="black", tags=f"key_{pitch}")
                self.key_map[pitch] = rect

    def set_active_keys(self, notes):
        #updatin
        for n in self.active_notes:
            if n not in notes and n in self.key_map:
                is_black = (n % 12) not in [0, 2, 4, 5, 7, 9, 11]
                color = "black" if is_black else "white"
                self.itemconfig(self.key_map[n], fill=color)

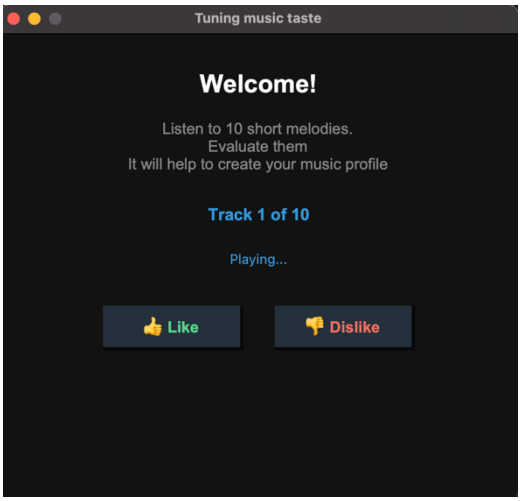
        for n in notes:
            if n in self.key_map:
                # pressing button color
                self.itemconfig(self.key_map[n], fill="#00ccff")

        self.active_notes = set(notes)
```

Code fragment 13.

PianoKeyboard is a melody visualization on a piano keyboard which displays which notes are currently playing in real time. The class inherits from tk.Canvas, allowing precise drawing and

color control. White keys are initially drawn and evenly spaced. Then, black keys are drawn relative to the white keys. Each key is mapped to its MIDI pitch number using a dictionary of key_map. The method “set_active_keys(notes)” resets previously active keys to their original color, highlights currently active notes with a particular color.



The program has a class of OnboardingWindow. It is the first interface shown to the user when the application starts. If there is no JSON profile of the user saved, this window is displayed. It plays 10 random melodies for the user to evaluate. By user’s likes and dislikes, the program creates the user’s music profile.

Photo 1. Onboarding window.

The App class is the central controller of the program. It is responsible for GUI and connecting all the components of the code into a single workflow.
It initializes UserProfile(), MelodyGenerator(), and MusicAnalyst() to set their interaction. The variables (self.current_score, self.current_features, self.current_bpm, self.note_timeline, self.is_playing) store the current music state, including the generated music21score, its analyzed vectors, tempo information, a precalculated note timeline for visualization, and a playback status.
The application uses pygame.mixer for MIDI playback. It allows looping playback, synchronization with the piano keyboard, and precise time querying.
Def ‘setup_ui” sets up the UI using installed Tkinter methods and custom Tkinter methods that I have previously created. In the left panel, there are controls, buttons, selectors, profile management. In the right panel, there are visualizations of the music profile and a piano keyboard.
The polar chart, created by Matplotlib, displays the current user profile and current track’s features. The PianoKeyboard widget displays active notes, visualizing the MIDI.
The UI includes “Generate” and “Random” buttons, “Play / Stop” playback controls, BPM input, mood override menu, instrument selection carousels, drum and bass toggles, like/dislike feedback buttons, export/reset controls, and an instruction button. Def “generate_music” interprets the selected mood override and maps it to a profile override dictionary. If “By my profile” is selected, the user profile is only used. The def “_run_generation” calls “MelodyGenerator.generate_track”, analyzes the resulting score with MusicAnalyst, extracts BPM from the score, builds a note timeline for piano, updates instruments, enables buttons for interacting, updates the radar chart, and starts playback.
Random generation is not primarily based on any preset, which allows exploration outside the learned preference space.
Before playback, the current score is written to a temporary MIDI file to withstand the manual changes of BPM and be compatible with pygame.mixer. Def “play_music” and “stop_music’

update UI buttons and piano animation with the playback. Def “animate_piano” synchronizes audio playback with visual playback” using precomputed “note_timeline” to avoid real-time analysis. Def “build_timeline” converts music to time intervals (ms) – melody and harmony parts are included – and is used for animating the piano. Def “update_current_instrument” changes the instrument according tp the carousel. Def “apply_bpm” allows manual tempo adjustment according to the Textbox in the GUI. Def “give_feedback” gives user feedback to UserProfile for further changing of the profile. Def “reset_profile” restores all vectors’ values to 0.5 (neutral). App class supports export in MIDI (direct), WAV (rendered via FluidSynth and Soundfont), and Musescore (MusicXML export and automatic opening). It regulates the opening of an onboarding window, checking the existence of the user profile in JSON.

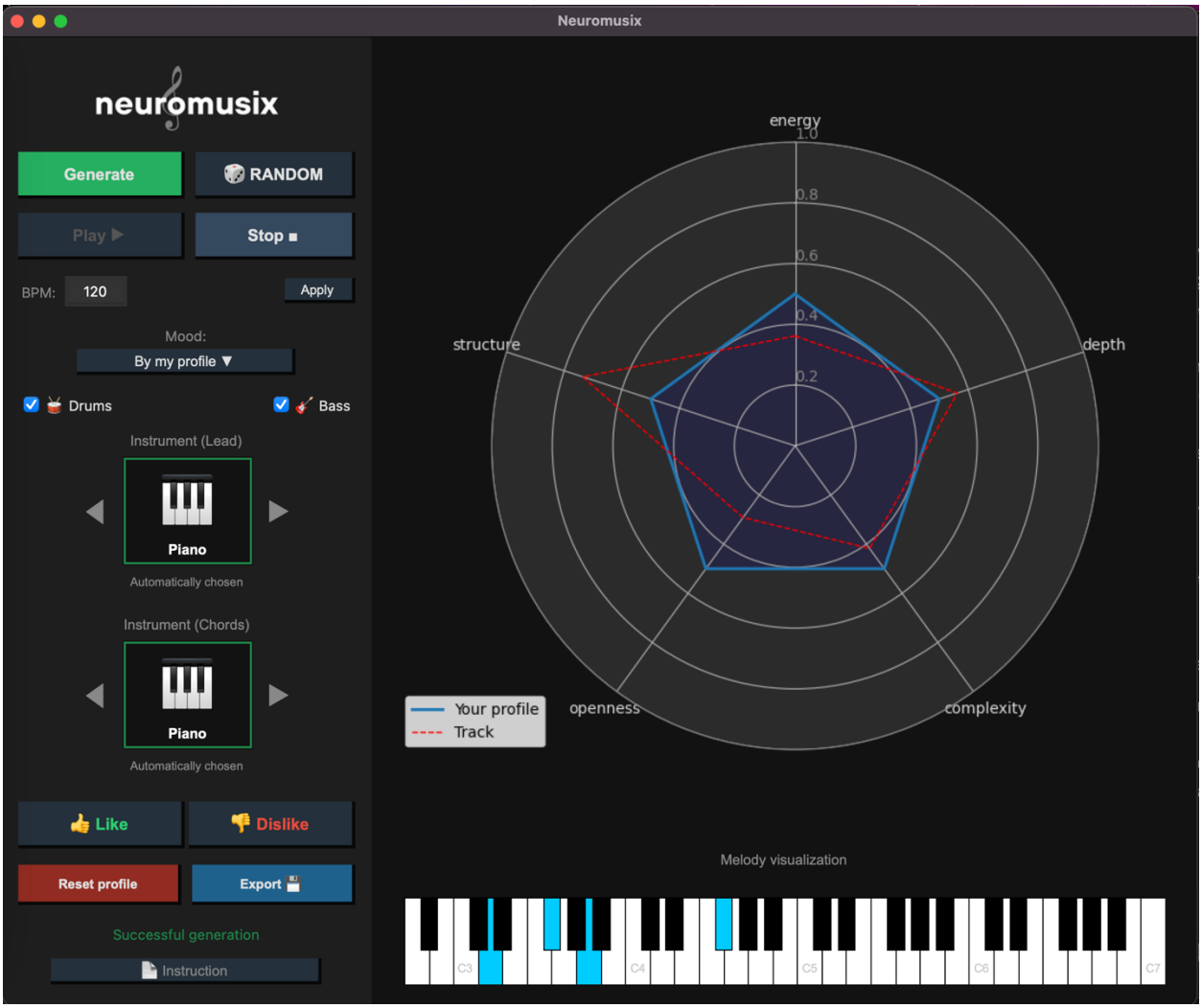
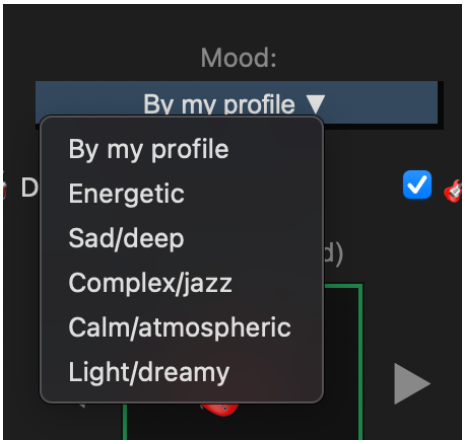
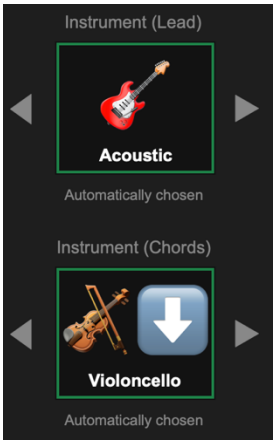


Photo 2. The GUI of Neuromusix.

Finally, the final application was created. In the left panel, there are logo, modification, playback, and export buttons. In the right panel, there is visualization of the radar chart and a piano keyboard.

The user can click “Generate” to generate a track, and it will start to play automatically. The melody will be based on the chosen mood, instruments in the carousel, and ticked “Drums” and “Bass” (if the user does not tick them, they will not play). On the radar chart, their profile’s vectors and track’s vectors are displayed. On the piano keyboard, the notes played are displayed (chords and lead melody). By clicking “Stop”, the music stops playing. The user can change the instruments and the BPM of the melody, then click “Play” again. By clicking “random”, the random music is generated. After clicking “Like” or “Dislike”, vectors of their profile are changed and the polar chart is adapted. By clicking “Reset profle”, the profile resets to neutral values. The “Export” button opens a window, in which the user can choose the export format (MIDI, WAV, MuseScore 4). There is a green label that indicates changes like “Successful generation” or “Music profile updated”. By clicking “Introduction”, a window of a TXT file is opened. The file explains the user how to use the program properly.





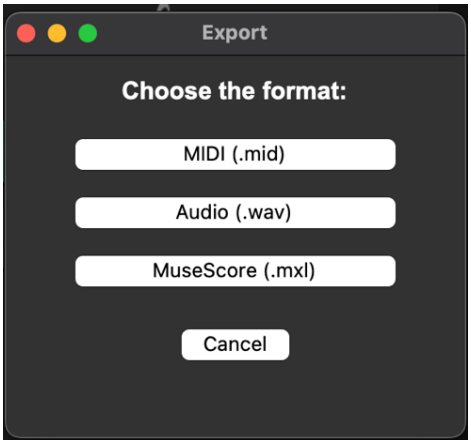


Photo 3, 4, 5. Mood button; Instrument carousel; Export window.

Conclusion

In conclusion, I would like to note that the step-by-step implementation of the initial tasks led to more efficient work on the implementation of my idea. A review of analogs allowed me to identify their shortcomings, which, as far as possible, I tried to take into account when creating the program. The programming language Python helped me to realize my ideas effectively and flexibly. The main advantage of Python was the presence of libraries for creating musical compositions, namely music21. Other languages do not have such advantages. These libraries and others (for working with notes, MIDI, lists, graphical interface) were studied and used during the project. I was able to get acquainted with music theory and reinforcement learning techniques that I have successfully implemented in my project. The theory was used to create the intelligence of my music generator, which learns by user's feedback. The reinforcement learning was a new direction for me: rather than using machine learning, RL made the program flexible and dynamic, and did not require massive data learning. Detailed work was carried out on the code by previously creating flowcharts of the architecture of my program.

My final product has advantages that were not observed in other analogues. The first advantage is the possibility of exporting to MIDI, MP3, and musical notation. The second advantage is the ability to customize instruments that play the melody. The third is that melody is generated by a user's music profile, which offers personalization. The program creates a coherent symphonic composition of different instruments and parts, which are customizable. It is important to notice that reinforcement learning was incredible for defining an individual user's taste. The program can be used by musicians to get inspiration for writing their own songs. Also, the product aligns with the Research of the American Music Therapy Association, as the music can adapt to a person's mood, resulting in a possibility of using such a product in music medicine.

It is worth considering that the product requires improvements. For instance, a drum part could be customized or changed, as it plays only one pattern on one instrument. A bass part could be customized, too because it plays only on a bass guitar. Some instruments are not present in the program due to music21 limitations, which can be changed by adding the possibility of input of user's own Soundfont of other instruments.

The mentor of the project (Ustinova Lyudmila Valeryevna, teacher of informatics) has provided feedback: "The project demonstrates a successful combination of theoretical knowledge and practical skills, a high level of independence, and a conscious choice of modern artificial intelligence technologies. The work has significant potential for further development".

References

[1] Musicful AI; AI Music Industry Statistics 2025: Market Size, Tools, and Trends; Online; Musicful.ai; 2025. Link: https://www.musicful.ai/news/ai-music-statistics/

[2] BeatsToRapOn; AI Music 2026: Global Adoption, Market Trends & Creative Evolution Outlook; Online; Beatstorapon.com; 2025. Link: https://beatstorapon.com/blog/ai-music-2026-global-adoption-market-trends-creative-evolution-outlook/

[3] WaveSpeedAI; Best AI Music Generators in 2026: Create Professional Audio with AI; Online; Wavespeed.ai; 2025. Link: https://wavespeed.ai/blog/posts/best-ai-music-generators-2026/

[4] Starcke K., Mayr J., von Georgi R.; Emotion Modulation through Music after Sadness Induction—The Iso Principle in a Controlled Experimental Study; Basel; International Journal of Environmental Research and Public Health; 2021, 18(23), 12486. Link: https://pmc.ncbi.nlm.nih.gov/articles/PMC8656869/

[5] Salimpoor V.N., Benovoy M., Larcher K., Dagher A., Zatorre R.J.; Anatomically distinct dopamine release during anticipation and experience of peak emotion to music; New York; Nature Neuroscience; 2011, 14(2), 257-262. Link: https://pubmed.ncbi.nlm.nih.gov/21217764/